

KilimoPRO 2.0 - Full Implementation

Production-ready agricultural intelligence platform for all 8 IGAD countries

🚀 Quick Start

Prerequisites

- Node.js 18+
- PostgreSQL 14+
- Twilio account (for SMS) or Africa's Talking

Setup

```bash

##### # 1. Clone and install

```
npx create-next-app@latest kilimopro-v2 --typescript --tailwind --eslint --app --src-dir=false
cd kilimopro-v2
npm install @radix-ui/react-select @radix-ui/react-tabs @tanstack/react-query axios drizzle-orm
pg postgres leaflet react-leaflet zod
npm install -D drizzle-kit @types/pg @types/leaflet
```

##### # 2. Configure database

createdb kilimopro

# Update .env with DATABASE\_URL=postgresql://user:password@localhost:5432/kilimopro

##### # 3. Set up Drizzle

npx drizzle-kit init

# Update drizzle.config.ts (see below)

npx drizzle-kit generate

##### # 4. Run migrations

npx drizzle-kit migrate

##### # 5. Seed data

npm run db:seed

##### # 6. Start server

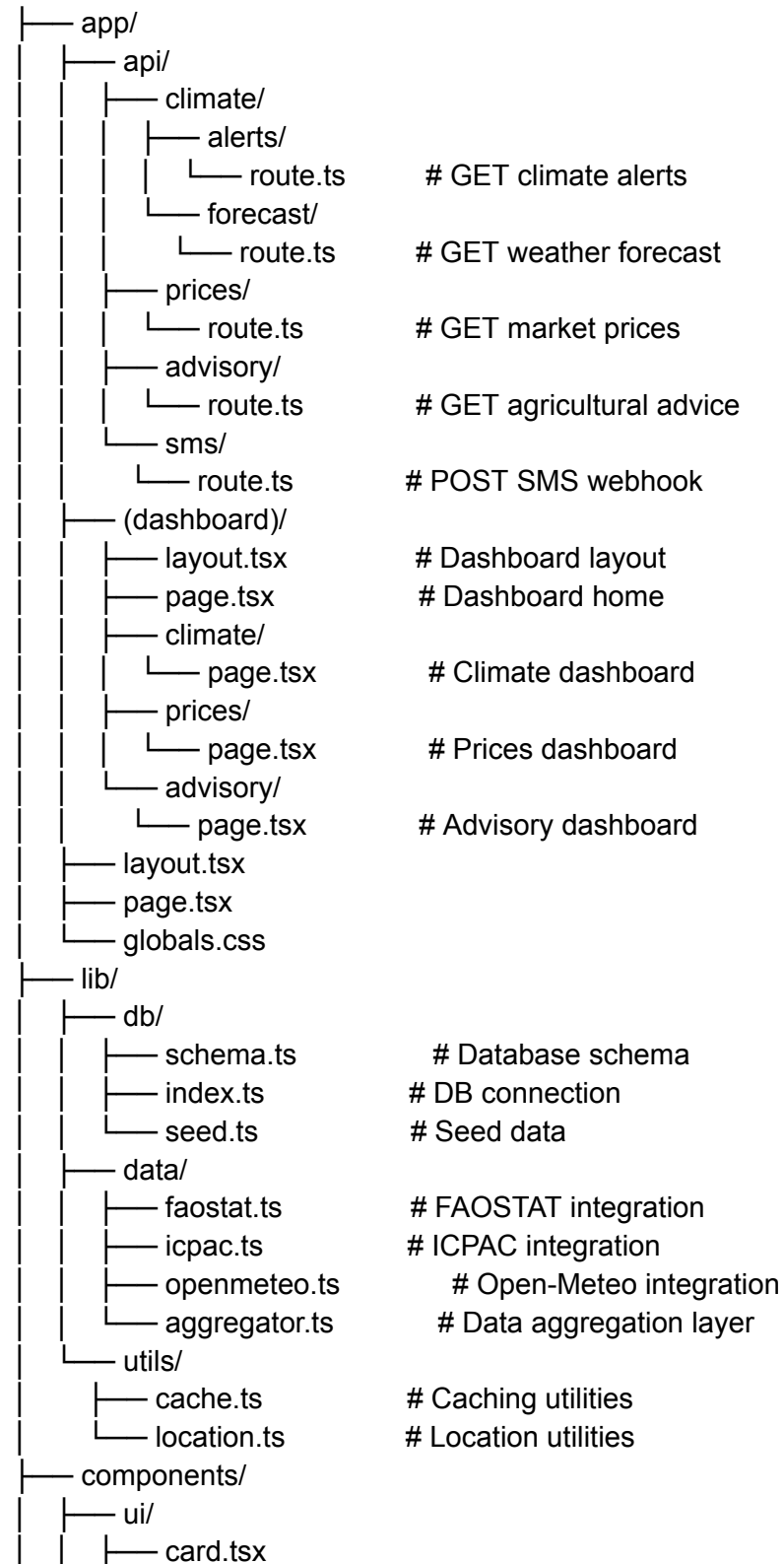
npm run dev

```

📁 Project Structure

...

kilimopro-v2/



```

|   |   |   | select.tsx
|   |   |   | tabs.tsx
|   |   |   |
|   |   |   | shared/
|   |   |   |   | climate-map.tsx
|   |   |   |
|   |   |   | types/
|   |   |   |   | index.ts
|   |   |   |
|   |   |   | drizzle/
|   |   |   |   | migrations/
|   |   |   |
|   |   |   | .env.example
|   |   |   | package.json
|   |   |   | tailwind.config.ts
|   |   |   | tsconfig.json
|   |   |   | drizzle.config.ts
|   |   |   |
|   |   |   | ...
|   |   |   |
|   |   |   | ---
|   |   |   |
|   |   |   | ## 📦 package.json
|   |   |   |
|   |   |   | ```json
|   |   |   | {
|   |   |   |   "name": "kilimopro-v2",
|   |   |   |   "version": "1.0.0",
|   |   |   |   "private": true,
|   |   |   |   "scripts": {
|   |   |   |     "dev": "next dev",
|   |   |   |     "build": "next build",
|   |   |   |     "start": "next start",
|   |   |   |     "lint": "next lint",
|   |   |   |     "db:generate": "drizzle-kit generate",
|   |   |   |     "db:migrate": "drizzle-kit migrate",
|   |   |   |     "db:seed": "tsx lib/db/seed.ts",
|   |   |   |     "db:studio": "drizzle-kit studio"
|   |   |   |   },
|   |   |   |   "dependencies": {
|   |   |   |     "@radix-ui/react-dropdown-menu": "^2.1.1",
|   |   |   |     "@radix-ui/react-select": "^2.1.1",
|   |   |   |     "@radix-ui/react-tabs": "^1.1.1",
|   |   |   |     "@tanstack/react-query": "^5.50.0",
|   |   |   |     "axios": "^1.7.2",
|   |   |   |     "drizzle-orm": "^0.30.4",
|   |   |   |     "leaflet": "^1.9.4",
|   |   |   |     "lucide-react": "^0.378.0",
|   |   |   |     "next": "14.2.3",

```

```

    "pg": "^8.11.3",
    "postgres": "^3.4.4",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-leaflet": "^4.2.1",
    "tailwind-merge": "^2.3.0",
    "tailwindcss-animate": "^1.0.7",
    "zod": "^3.23.8"
  },
  "devDependencies": {
    "@types/leaflet": "^1.9.12",
    "@types/node": "^20.12.12",
    "@types/pg": "^8.11.6",
    "@types/react": "^18.3.3",
    "@types/react-dom": "^18.3.0",
    "autoprefixer": "^10.4.19",
    "drizzle-kit": "^0.20.17",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.3",
    "typescript": "^5.4.5",
    "tsx": "^4.16.2"
  }
}
...

```

🗄 Database Schema

lib/db/schema.ts

```

``typescript
import { pgTable, serial, varchar, text, timestamp, date, decimal, integer, jsonb, boolean } from
'drizzle-orm/pg-core';

// =====
// CORE TABLES
// =====

// IGAD Countries
export const countries = pgTable('countries', {
  id: serial('id').primaryKey(),
  code: varchar('code', { length: 2 }).unique().notNull(),
  name: varchar('name', { length: 100 }).notNull(),

```

```
igadMember: boolean('igad_member').default(false),
flag: varchar('flag', { length: 5 }),
createdAt: timestamp('created_at').defaultNow(),
});
```

```
// Regions/Provinces/States
```

```
export const regions = pgTable('regions', {
  id: serial('id').primaryKey(),
  countryId: integer('country_id').references(() => countries.id),
  code: varchar('code', { length: 10 }).notNull(),
  name: varchar('name', { length: 100 }).notNull(),
  latitude: decimal('latitude', { precision: 10, scale: 7 }),
  longitude: decimal('longitude', { precision: 10, scale: 7 }),
  createdAt: timestamp('created_at').defaultNow(),
});
```

```
// Crops
```

```
export const crops = pgTable('crops', {
  id: serial('id').primaryKey(),
  code: varchar('code', { length: 10 }).unique().notNull(),
  name: varchar('name', { length: 100 }).notNull(),
  scientificName: varchar('scientific_name', { length: 100 }),
  category: varchar('category', { length: 50 }),
  faostatCode: varchar('faostat_code', { length: 10 }),
  createdAt: timestamp('created_at').defaultNow(),
});
```

```
// Markets
```

```
export const markets = pgTable('markets', {
  id: serial('id').primaryKey(),
  regionId: integer('region_id').references(() => regions.id),
  name: varchar('name', { length: 100 }).notNull(),
  latitude: decimal('latitude', { precision: 10, scale: 7 }),
  longitude: decimal('longitude', { precision: 10, scale: 7 }),
  type: varchar('type', { length: 50 }), // wholesale, retail, farmgate
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
});
```

```
// =====
// DATA TABLES
// =====
```

```
// Market Prices (cached from FAOSTAT)
```

```

export const marketPrices = pgTable('market_prices', {
  id: serial('id').primaryKey(),
  marketId: integer('market_id').references(() => markets.id),
  cropId: integer('crop_id').references(() => crops.id),
  countryCode: varchar('country_code', { length: 2 }).notNull(),
  date: date('date').notNull(),
  price: decimal('price', { precision: 12, scale: 2 }).notNull(),
  priceMin: decimal('price_min', { precision: 12, scale: 2 }),
  priceMax: decimal('price_max', { precision: 12, scale: 2 }),
  unit: varchar('unit', { length: 20 }).notNull(),
  currency: varchar('currency', { length: 3 }).notNull(),
  source: varchar('source', { length: 50 }).notNull(),
  isVerified: boolean('is_verified').default(false),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

```

```

// Climate Data (cached from ICPAC, Open-Meteo)
export const climateData = pgTable('climate_data', {
  id: serial('id').primaryKey(),
  regionId: integer('region_id').references(() => regions.id),
  countryCode: varchar('country_code', { length: 2 }).notNull(),
  date: date('date').notNull(),
  temperatureMin: decimal('temperature_min', { precision: 5, scale: 2 }),
  temperatureMax: decimal('temperature_max', { precision: 5, scale: 2 }),
  temperatureAvg: decimal('temperature_avg', { precision: 5, scale: 2 }),
  rainfall: decimal('rainfall', { precision: 8, scale: 2 }),
  humidity: decimal('humidity', { precision: 5, scale: 2 }),
  windSpeed: decimal('wind_speed', { precision: 5, scale: 2 }),
  soilMoisture: decimal('soil_moisture', { precision: 5, scale: 2 }),
  source: varchar('source', { length: 50 }).notNull(),
  createdAt: timestamp('created_at').defaultNow(),
});

```

```

// Hazard Alerts (from ICPAC)
export const hazardAlerts = pgTable('hazard_alerts', {
  id: serial('id').primaryKey(),
  regionId: integer('region_id').references(() => regions.id),
  type: varchar('type', { length: 50 }).notNull(), // drought, flood, pest, rainfall, extreme_rainfall
  severity: varchar('severity', { length: 20 }).notNull(), // low, moderate, high, extreme
  title: varchar('title', { length: 200 }).notNull(),
  description: text('description'),
  affectedCountries: jsonb('affected_countries').notNull(),
  affectedRegions: jsonb('affected_regions'),
});

```

```

    startDate: date('start_date').notNull(),
    endDate: date('end_date'),
    source: varchar('source', { length: 50 }).notNull(),
    isActive: boolean('is_active').default(true),
    createdAt: timestamp('created_at').defaultNow(),
    updatedAt: timestamp('updated_at').defaultNow(),
  });

```

```

// Agriculture Watch (from ICPAC)
export const agricultureWatch = pgTable('agriculture_watch', {
  id: serial('id').primaryKey(),
  date: date('date').notNull(),
  summary: text('summary').notNull(),
  cropConditions: jsonb('crop_conditions').notNull(),
  rangelandConditions: jsonb('rangeland_conditions').notNull(),
  rainfallAnomalies: jsonb('rainfall_anomalies'),
  soilMoisture: jsonb('soil_moisture'),
  vegetationIndex: jsonb('vegetation_index'),
  source: varchar('source', { length: 50 }).notNull(),
  createdAt: timestamp('created_at').defaultNow(),
});

```

```

// Advisory Content
export const advisoryContent = pgTable('advisory_content', {
  id: serial('id').primaryKey(),
  regionId: integer('region_id').references(() => regions.id),
  cropId: integer('crop_id').references(() => crops.id),
  countryCode: varchar('country_code', { length: 2 }),
  type: varchar('type', { length: 50 }).notNull(), // planting, harvesting, fertilizer, pest_control,
  irrigation
  title: varchar('title', { length: 200 }).notNull(),
  content: text('content').notNull(),
  language: varchar('language', { length: 10 }).notNull(),
  startDate: date('start_date'),
  endDate: date('end_date'),
  season: varchar('season', { length: 50 }), // long_rains, short_rains, dry_season
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

```

```

// =====
// USER & SMS TABLES
// =====

```

// Users

```
export const users = pgTable('users', {
  id: serial('id').primaryKey(),
  phoneNumber: varchar('phone_number', { length: 20 }).unique(),
  countryId: integer('country_id').references(() => countries.id),
  regionId: integer('region_id').references(() => regions.id),
  language: varchar('language', { length: 10 }).default('en'),
  preferredCrops: jsonb('preferred_crops'),
  farmSize: decimal('farm_size', { precision: 10, scale: 2 }),
  farmSizeUnit: varchar('farm_size_unit', { length: 10 }),
  createdAt: timestamp('created_at').defaultNow(),
  lastActiveAt: timestamp('last_active_at'),
});
```

// SMS Logs

```
export const smsLogs = pgTable('sms_logs', {
  id: serial('id').primaryKey(),
  userId: integer('user_id').references(() => users.id),
  phoneNumber: varchar('phone_number', { length: 20 }).notNull(),
  message: text('message').notNull(),
  direction: varchar('direction', { length: 10 }).notNull(), // in, out
  status: varchar('status', { length: 20 }).notNull(), // sent, delivered, failed, read
  gateway: varchar('gateway', { length: 50 }), // twilio, africastalking, ussd
  gatewayMessageId: varchar('gateway_message_id', { length: 100 }),
  cost: decimal('cost', { precision: 10, scale: 4 }),
  createdAt: timestamp('created_at').defaultNow(),
});
```

// USSD Sessions

```
export const ussdSessions = pgTable('ussd_sessions', {
  id: serial('id').primaryKey(),
  phoneNumber: varchar('phone_number', { length: 20 }).notNull(),
  sessionId: varchar('session_id', { length: 100 }).notNull(),
  network: varchar('network', { length: 50 }),
  currentMenu: varchar('current_menu', { length: 100 }),
  menuHistory: jsonb('menu_history'),
  userInput: jsonb('user_input'),
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});
```

// =====


```
// CACHE & LOGGING
```

```
// =====
```

```
// Cache for API responses
```

```
export const cache = pgTable('cache', {  
  id: serial('id').primaryKey(),  
  key: varchar('key', { length: 255 }).unique().notNull(),  
  value: jsonb('value').notNull(),  
  expiresAt: timestamp('expires_at').notNull(),  
  createdAt: timestamp('created_at').defaultNow(),  
});
```

```
// API Request Logs
```

```
export const apiLogs = pgTable('api_logs', {  
  id: serial('id').primaryKey(),  
  endpoint: varchar('endpoint', { length: 100 }).notNull(),  
  method: varchar('method', { length: 10 }).notNull(),  
  status: integer('status').notNull(),  
  requestData: jsonb('request_data'),  
  responseData: jsonb('response_data'),  
  error: text('error'),  
  ipAddress: varchar('ip_address', { length: 50 }),  
  userAgent: text('user_agent'),  
  durationMs: integer('duration_ms'),  
  createdAt: timestamp('created_at').defaultNow(),  
});
```

```
// =====
```

```
// EXPORT ALL
```

```
// =====
```

```
export type Country = typeof countries.$inferSelect;  
export type Region = typeof regions.$inferSelect;  
export type Crop = typeof crops.$inferSelect;  
export type Market = typeof markets.$inferSelect;  
export type MarketPrice = typeof marketPrices.$inferSelect;  
export type ClimateDatum = typeof climateData.$inferSelect;  
export type HazardAlert = typeof hazardAlerts.$inferSelect;  
export type AgricultureWatch = typeof agricultureWatch.$inferSelect;  
export type AdvisoryContent = typeof advisoryContent.$inferSelect;  
export type User = typeof users.$inferSelect;  
export type SmsLog = typeof smsLogs.$inferSelect;  
export type UssdSession = typeof ussdSessions.$inferSelect;
```

```
export type NewCountry = typeof countries.$inferInsert;
export type NewRegion = typeof regions.$inferInsert;
export type NewCrop = typeof crops.$inferInsert;
export type NewMarket = typeof markets.$inferInsert;
export type NewMarketPrice = typeof marketPrices.$inferInsert;
export type NewClimateDatum = typeof climateData.$inferInsert;
export type NewHazardAlert = typeof hazardAlerts.$inferInsert;
export type NewAgricultureWatch = typeof agricultureWatch.$inferInsert;
export type NewAdvisoryContent = typeof advisoryContent.$inferInsert;
export type NewUser = typeof users.$inferInsert;
export type NewSmsLog = typeof smsLogs.$inferInsert;
export type NewUssdSession = typeof ussdSessions.$inferInsert;
```